

МИНОБРНАУКИ РОССИИ
Федеральное государственное автономное образовательное учреждение высшего
образования
«Московский государственный технологический университет «СТАНКИН»
(ФГАОУ ВО «МГТУ «СТАНКИН»)

Кафедра Центр подготовки топ-специалистов в сфере ИТ

ЛАБОРАТОРНЫЙ ПРАКТИКУМ

по дисциплине

«Технологии разработки программных средств»

для направления 09.03.02 «Информационные системы и технологии»
направленность «Разработка и внедрение корпоративных информационных систем»

Автор: Семенов А.С.

Москва, 2026 г.

Содержание

Введение. Сквозной проект и карта переходов	2
1 Варианты сквозных проектов (инструменты для разработчиков)	4
2 Лабораторная работа №1. Анализ требований и структурное проектирование	7
2.1 Цель	7
2.2 Теоретические сведения	7
2.3 Задание (для любого варианта)	7
2.4 Пример DFD (фрагмент для «Редактора ER-диаграмм»)	7
2.5 Пример ER-диаграммы для инструмента	7
2.6 Требования к отчёту	8
2.7 Критерии оценки (20 баллов)	8
3 Лабораторная работа №2. Объектно-ориентированный анализ и проектирование (UML)	9
3.1 Цель	9
3.2 Теоретические сведения	9
3.3 Задание (для любого варианта)	9
3.4 Пример диаграммы классов (для «Редактора ER-диаграмм»)	9
3.5 Требования к отчёту	9
3.6 Критерии оценки (20 баллов)	9
4 Лабораторная работа №3. Разработка прототипа, CI/CD и контейнеризация	11
4.1 Цель	11
4.2 Теоретические сведения	11
4.3 Задание (для любого варианта)	11
4.4 Пример фрагмента кода (FastAPI)	11
4.5 Пример Dockerfile	11
4.6 Критерии оценки (25 баллов)	12
5 Лабораторная работа №4. Тестирование и мониторинг	13
5.1 Цель	13
5.2 Теоретические сведения	13
5.3 Задание	13
5.4 Пример теста (pytest)	13
5.5 Пример интеграционного теста с testcontainers	13
5.6 Критерии оценки (25 баллов)	13
6 Лабораторная работа №5. Интеграция AI-ассистента и цифрового двойника	14
6.1 Цель	14
6.2 Теоретические сведения	14
6.3 Задание	14
6.4 Пример обучения модели	14
6.5 Пример маршрута /predict	14
6.6 Критерии оценки (25 баллов)	15
Заключение	16
Приложение А. Шаблон отчёта	17

Введение. Сквозной проект и карта переходов между индустриями

Общая концепция

В рамках лабораторного практикума студенты выполняют **сквозной проект** – разработку инструмента, предназначенного для поддержки процессов разработки ПО. В качестве предметной области выступает не бизнес-система, а **программный инструмент**, который используется аналитиками, архитекторами, разработчиками, тестировщиками или DevOps-инженерами. Это позволяет применить все изучаемые технологии (DFD, UML, REST API, CI/CD, тестирование, мониторинг, AI) для создания реального артефакта, востребованного в индустрии.

Каждый студент (или команда) получает **один из 30 вариантов** – конкретный тип инструмента. Все работы (№1–№5) выполняются в рамках выбранного инструмента.

Связь с эпохами и инструментами

В таблице показано, как каждая лабораторная работа соответствует эпохе, инструментам и ролям.

Таблица 1: Карта переходов между индустриями, инструментами и ролями

Работа	Индустрия	Инструменты	Роли	Стек технологий
№1	3.0 (автоматизация)	draw.io, MS Word	Аналитик, Архитектор	DFD, ERD, ТЗ по ГОСТ
№2	4.0 (цифровизация)	StarUML, PlantUML	Системный аналитик, Проектировщик	UML, DDD
№3	4.0–5.0	Git, Docker, GitHub Actions	Разработчик, DevOps-инженер	FastAPI, SQLAlchemy, Docker, CI/CD
№4	4.0–5.0	pytest, testcontainers, Prometheus, Grafana	QA-инженер, DevOps	тестирование, мониторинг, healthcheck
№5	5.0 (человекоцентричность)	GitHub Copilot, scikit-learn, pandas	ML-инженер, Разработчик	линейная регрессия, цифровой двойник

Архитектурная схема сквозного проекта

Общая архитектура инструмента (независимо от варианта) предполагает:

- Backend – REST API (FastAPI) для управления данными и настройками.
- База данных (SQLite/PostgreSQL) для хранения метаданных, проектов, диаграмм, требований и т.п.
- Фронтенд – может быть реализован в виде веб-интерфейса, консольного клиента или интеграции с IDE.
- Модуль AI/ML – для прогнозирования, рекомендаций, автоматической генерации.
- Инфраструктура: Docker, CI/CD, мониторинг.

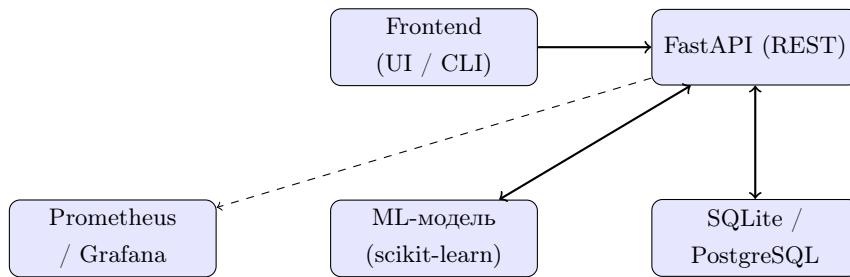


Рис. 1: Архитектура сквозного проекта

Инструментальный стек проекта

- **Языки:** Python (FastAPI, SQLAlchemy), SQL, YAML (docker-compose, GitHub Actions)
- **Инструменты разработки:** Git, GitHub, VS Code, Docker
- **CI/CD:** GitHub Actions
- **Тестирование:** pytest, testcontainers
- **Мониторинг:** Prometheus, Grafana
- **ML и AI:** scikit-learn, pandas, GitHub Copilot
- **Моделирование:** draw.io, StarUML, PlantUML

1 Варианты сквозных проектов (инструменты для разработчиков)

Каждый вариант представляет собой **конкретный программный инструмент**, который студент должен разработать в ходе выполнения всех пяти лабораторных работ. Выбор варианта осуществляется по указанию преподавателя или по желанию студента.

Таблица 2: 30 вариантов сквозных проектов

№	Название инструмента	Краткое описание	Основные функции / особенности
1	Редактор диаграмм ER-	Визуальное построение ER-диаграмм, экспорт в SQL	Создание сущностей, связей, атрибутов, генерация DDL, синхронизация с БД
2	Генератор кода по UML-классам	По диаграмме классов генерирует код (Python/Java)	Парсинг XMI, шаблоны кодогенерации, настройка стиля, обратное проектирование
3	Средство построения DFD	Редактор диаграмм потоков данных с проверкой целостности	Нотация Гейна-Сарсона, валидация, экспорт в PNG, иерархия уровней
4	Автоматический генератор тестов	Генерация модульных тестов для Python/Java	Анализ кода, рефлексия, генерация тестовых данных, поддержка pytest/JUnit
5	Система контроля версий (упрощённый Git)	Работа с текстовыми файлами: сравнение, слияние, история	Деревья версий, diff-алгоритм, коммиты, ветки, консольный интерфейс
6	CI/CD-платформа (мини-аналог Jenkins)	Управление пайплайнами сборки и тестирования	Оркестрация контейнеров, логирование, управление окружениями, Web UI
7	Средство мониторинга (Prometheus-подобное)	Сбор метрик, алертинг, визуализация	Экспорт метрик, правила алертинга, дашборды, интеграция с Grafana
8	Анализатор зависимостей в коде	Граф зависимостей между модулями	Парсинг импортов, поиск циклических зависимостей, визуализация графа
9	Редактор диаграмм последовательности	UML Sequence diagram, экспорт в PlantUML	Добавление объектов, сообщений, фрагментов, валидация синтаксиса
10	Генератор документации по коду	Анализ исходников, создание HTML/PDF документации	Парсинг docstring, шаблоны, поддержка Python/Java, интеграция с CI
11	Система управления требованиями	Хранение, трассировка и связывание требований	CRUD требований, связи с тестами, отчёты, импорт/экспорт Excel
12	Редактор диаграмм деятельности	UML Activity Diagram, симуляция выполнения	Визуальное создание процессов, проверка на deadlock, экспорт в BPMN
13	Инструмент reverse engineering	Восстановление UML-диаграмм по коду	Парсинг AST, построение Class и Sequence, интеграция с IDE

Таблица 2 – продолжение

№	Название инструмента	Краткое описание	Основные функции / особенности
14	Платформа рефакторинга	Поиск и исправление «запахов» кода	Анализ копиясты, длинных методов, автоматическое применение исправлений
15	Система управления конфигурациями	Централизованное хранение конфигов для микросервисов	REST API, версионирование, уведомления об изменениях, поддержка YAML/JSON
16	Средство профилирования	Измерение времени выполнения методов, флеймграфы	Инструментирование, агрегация, визуализация в реальном времени
17	Генератор Use Case диаграмм	По текстовому описанию создаёт диаграмму вариантов использования	NLP-парсинг, генерация UML, экспорт, поддержка акторов
18	Линтер кода (статический анализатор)	Проверка стиля и потенциальных ошибок	Правила (PEP8, Checkstyle), отчёты, интеграция с CI
19	Редактор диаграмм классов с кодогенерацией	Визуальное проектирование классов и генерация кода	Обратное проектирование, шаблоны, экспорт в разные языки
20	Средство тестирования API (Postman-подобное)	Отправка запросов, коллекции, автотесты	REST клиент, переменные, генерация документации, выполнение тестов
21	Инструмент визуализации логов (ELK-аналог)	Приём логов, поиск, фильтрация, графики	Сбор из файлов/сокетов, индексация, дашборды, алертинг
22	Генератор бэклога (Agile-проекты)	Автоматическое создание User Stories на основе шаблонов	NLP, шаблоны, генерация, поддержка Scrum
23	Средство моделирования данных	Сравнение схем БД, генерация миграций	Сравнение, создание миграций, визуализация схем
24	Платформа управления деплоем (аналог Ansible)	Автоматизация развёртывания на удалённые серверы	SSH, сценарии, управление окружениями, роли
25	Анализатор безопасности (SAST)	Поиск уязвимостей в коде (SQLi, XSS, и др.)	Статический анализ, правила безопасности, отчёты
26	Редактор диаграмм состояний (Statechart)	Визуальное проектирование автоматов, кодогенерация	Нотация UML, проверка, симуляция, генерация Python/Java
27	Генератор архитектурных представлений	По коду строит C4-модель или слоистую архитектуру	Анализ структуры, визуализация, экспорт в Draw.io
28	Платформа для ревью кода	Создание ревью, комментарии, отслеживание	Интеграция с Git, управление замечаниями, статистика
29	Инструмент декомпозиции задач (Epic splitting)	Разбиение крупных задач на подзадачи с AI	NLP, генерация подзадач, оценка сложности
30	Цифровой двойник процесса разработки	Модель проекта, прогнозирование срывов сроков	Сбор метрик, прогнозирование, визуализация состояния

Как работать с вариантами

1. Студент выбирает (или получает) один вариант из таблицы.
2. Все пять лабораторных работ выполняются в контексте этого инструмента.
3. В каждой работе необходимо адаптировать общие требования к специфике выбранного инструмента (например, если вариант «Редактор ER-диаграмм», то в работе №1 нужно построить DFD и ERD для самого редактора, а не для бизнес-системы; в работе №2 – UML для редактора; в работе №3 – REST API для управления диаграммами; и т.д.).

2 Лабораторная работа №1. Анализ требований и структурное проектирование

2.1 Цель

Освоить методы структурного анализа: сбор требований, построение DFD и ER-диаграмм, оформление ТЗ для выбранного инструмента.

2.2 Теоретические сведения

Структурный анализ (эпоха 3.0) основан на функциональной декомпозиции. Основные нотации:

- **DFD** – потоки данных между процессами, сущностями, хранилищами.
- **ERD** – сущности, атрибуты, связи.

2.3 Задание (для любого варианта)

1. Проведите «интервью» с заказчиком (преподавателем) для уточнения функций вашего инструмента.
2. Составьте список функциональных и нефункциональных требований (не менее 10 и 5 соответственно).
3. Постройте контекстную диаграмму DFD и выполните декомпозицию до 2–3 уровней для ключевых процессов.
4. Постройте ER-диаграмму для хранения данных вашего инструмента.
5. Оформите техническое задание (ТЗ) по ГОСТ 34.602-89 или упрощённому шаблону.
6. Подготовьте отчёт.

2.4 Пример DFD (фрагмент для «Редактора ER-диаграмм»)

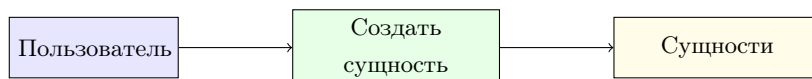


Рис. 2: Фрагмент DFD: создание сущности в редакторе ER-диаграмм

2.5 Пример ER-диаграммы для инструмента

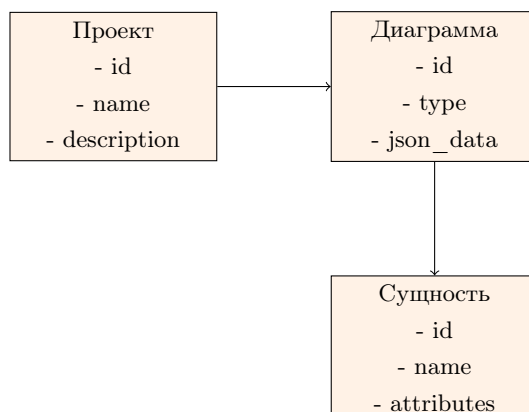


Рис. 3: Пример ER-диаграммы для хранения данных инструмента

2.6 Требования к отчёту

Отчёт должен содержать: титул, цель, задание, список требований, DFD (контекстная + декомпозиция), ER-диаграмму, текст ТЗ, выводы.

2.7 Критерии оценки (20 баллов)

- Полнота требований – 5
- Качество DFD (правильность, декомпозиция) – 5
- Корректность ER-диаграммы – 4
- Оформление ТЗ (соответствие структуре) – 4
- Качество отчёта – 2

3 Лабораторная работа №2. Объектно-ориентированный анализ и проектирование (UML)

3.1 Цель

Освоить UML для моделирования выбранного инструмента и применить принципы DDD.

3.2 Теоретические сведения

В 4.0 используется объектно-ориентированный подход. Основные диаграммы: Use Case, Class, Sequence. DDD выделяет агрегаты, ограниченные контексты.

3.3 Задание (для любого варианта)

1. Постройте диаграмму вариантов использования (Use Case) для вашего инструмента.
2. Постройте диаграмму классов (ключевые сущности и связи).
3. Разработайте диаграмму последовательности для ключевого сценария (например, «сохранение диаграммы», «генерация кода» и т.п.).
4. Выделите ограниченные контексты и агрегаты по DDD.
5. Подготовьте отчёт.

3.4 Пример диаграммы классов (для «Редактора ER-диаграмм»)

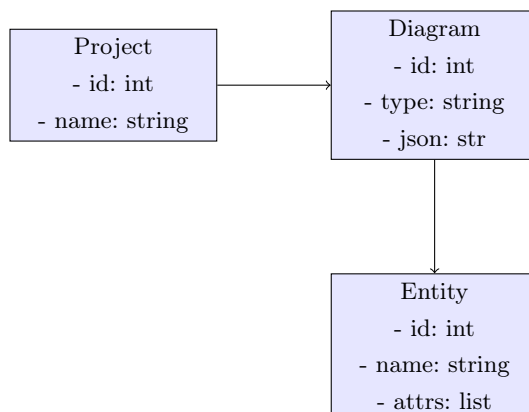


Рис. 4: Фрагмент диаграммы классов для инструмента

3.5 Требования к отчёту

Отчёт должен содержать диаграммы UML (Use Case, Class, Sequence) и описание ограниченных контекстов/агрегатов.

3.6 Критерии оценки (20 баллов)

- Use Case – 5
- Class diagram – 6
- Sequence diagram – 4

- DDD (контексты, агрегаты) – 3
- Оформление – 2

4 Лабораторная работа №3. Разработка прототипа, CI/CD и контейнеризация

4.1 Цель

Получить навыки разработки REST API (микросервиса), работы с Git, Docker и CI/CD.

4.2 Теоретические сведения

FastAPI – современный Python-фреймворк. Docker упаковывает приложение с зависимостями. GitHub Actions автоматизирует сборку и тесты.

4.3 Задание (для любого варианта)

1. Реализуйте REST API (FastAPI) для управления данными вашего инструмента (например, CRUD для проектов, диаграмм, сущностей).
2. Используйте SQLAlchemy и SQLite (или PostgreSQL).
3. Создайте Git-репозиторий, выполните не менее 5 осмысленных коммитов.
4. Напишите Dockerfile и docker-compose.yml для запуска приложения с БД.
5. Настройте GitHub Actions (сборка, запуск тестов, сборка Docker-образа).
6. Подготовьте отчёт.

4.4 Пример фрагмента кода (FastAPI)

Листинг 1: Маршрут создания проекта

```
1 @app.post("/projects/")
2 def create_project(project: ProjectCreate, db: Session = Depends(get_db)):
3     db_project = Project(**project.dict())
4     db.add(db_project)
5     db.commit()
6     db.refresh(db_project)
7     return db_project
```

4.5 Пример Dockerfile

Листинг 2: Dockerfile

```
1 FROM python:3.10
2 WORKDIR /app
3 COPY requirements.txt .
4 RUN pip install -r requirements.txt
5 COPY . .
6 CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

4.6 Критерии оценки (25 баллов)

- Работоспособность API – 6
- Dockerfile и docker-compose – 5
- CI/CD (успешный пайплайн) – 6
- История Git – 4
- Отчёт – 4

5 Лабораторная работа №4. Тестирование и мониторинг

5.1 Цель

Освоить модульное и интеграционное тестирование, настройку мониторинга с Prometheus и Grafana.

5.2 Теоретические сведения

pytest – фреймворк для тестирования. Testcontainers позволяет запускать контейнеры с БД для интеграционных тестов. Prometheus собирает метрики, Grafana визуализирует.

5.3 Задание

1. Напишите модульные тесты (pytest) для ключевых эндпоинтов (не менее 10 тестов).
2. Напишите интеграционные тесты с testcontainers (PostgreSQL) для проверки взаимодействия с БД.
3. Добавьте сбор метрик (prometheus-client) и эндпоинт /metrics.
4. Настройте Prometheus и Grafana (в контейнерах) для визуализации метрик, создайте дашборд.
5. Добавьте healthcheck-эндпоинт (/health).
6. Подготовьте отчёт.

5.4 Пример теста (pytest)

Листинг 3: Модульный тест для маршрута

```
1 def test_create_project(client):
2     response = client.post("/projects/", json={"name": "Test"})
3     assert response.status_code == 200
4     assert response.json()["name"] == "Test"
```

5.5 Пример интеграционного теста с testcontainers

Листинг 4: Интеграционный тест

```
1 from testcontainers.postgres import PostgresContainer
2
3 def test_db_connection():
4     with PostgresContainer("postgres:13") as postgres:
5         engine = create_engine(postgres.get_connection_url())
6         assert engine.connect()
```

5.6 Критерии оценки (25 баллов)

- Покрытие тестами (не менее 10 тестов) – 6
- Интеграционные тесты – 5
- Сбор метрик и Prometheus – 6
- Дашборд в Grafana – 4
- Healthcheck и отчёт – 4

6 Лабораторная работа №5. Интеграция AI-ассистента и цифрового двойника

6.1 Цель

Использовать AI-ассистент (Copilot) и создать модель прогнозирования или рекомендаций (цифровой двойник) для вашего инструмента.

6.2 Теоретические сведения

GitHub Copilot генерирует код по комментариям. Цифровой двойник – модель машинного обучения для прогнозирования (например, времени выполнения задачи, спроса на данные, загрузки системы и т.п.).

6.3 Задание

1. Используйте GitHub Copilot (или аналог) для генерации части кода (например, эндпоинта `/predict`, `/recommend`).
2. Соберите синтетические или реальные данные, характерные для вашего инструмента, и обучите модель линейной регрессии (или другую простую модель) для прогнозирования некоторой целевой переменной (время генерации кода, количество связей в диаграмме и т.п.).
3. Интегрируйте модель в приложение (добавьте эндпоинт `/predict` или `/recommend`).
4. Визуализируйте прогноз (например, на веб-странице или в виде графика).
5. Подготовьте отчёт.

6.4 Пример обучения модели

Листинг 5: Обучение линейной регрессии

```
1 import pandas as pd
2 from sklearn.linear_model import LinearRegression
3 from sklearn.metrics import r2_score
4
5 data = pd.read_csv("data.csv") # columns: feature1, feature2, target
6 X = data[["feature1", "feature2"]]
7 y = data["target"]
8 model = LinearRegression().fit(X, y)
9 print("R2:", r2_score(y, model.predict(X)))
```

6.5 Пример маршрута `/predict`

Листинг 6: Маршрут прогнозирования

```
1 @app.post("/predict")
2 def predict(features: dict):
3     # features: {"feature1": value1, "feature2": value2}
4     pred = loaded_model.predict([[features["feature1"], features["feature2"]]])
5     return {"predicted_value": pred[0]}
```

6.6 Критерии оценки (25 баллов)

- Использование Copilot (доказательства) – 5
- Качество модели (метрики) – 7
- Интеграция модели – 7
- Визуализация – 4
- Отчёт – 2

Заключение

Выполнив все пять лабораторных работ, студент создаёт работающий прототип инструмента для разработчиков, проходя полный цикл разработки – от анализа требований до интеграции AI. Проект охватывает три индустриальные эпохи (3.0, 4.0, 5.0), что даёт целостное понимание эволюции технологий, инструментов и ролей. Полученные навыки являются основой для дальнейшего профессионального развития в области программной инженерии.

Приложение А. Шаблон отчёта

Структура отчёта для каждой лабораторной работы:

1. Титульный лист (с указанием номера работы и варианта)
2. Цель работы
3. Задание
4. Теоретическая часть
5. Ход выполнения (с диаграммами, листингами кода, скриншотами)
6. Результаты
7. Выводы
8. Список использованных источников (если есть)

Приложение Б. Сводная таблица баллов

Работа	Макс. балл
№1	20
№2	20
№3	25
№4	25
№5	25
Итого	115

Для зачёта необходимо набрать ≥ 69 баллов (60% от максимума).